

WIT - NXP BATCH - IV
SYNOPSIS SUBMISSION

Batch Name - WIT Batch IV

Name of the students -

- Mehak Taneja
- Vanshika Gupta
- Varsha S
- Varshini G
- Yerukonda Venkata Lakshmi

Title of Project / Synopsis -

RTL Design of Dual-Port RAM using Verilog HDL

Guide Name -

Submission Date - 31/08/2026

1. Introduction

The project presents the RTL design and functional verification of a synchronous dual-port Random Access Memory (RAM) using Verilog HDL. A dual-port RAM is a memory architecture that provides two independent access ports, allowing read and write operations to be performed through Port A and Port B. This architecture is useful in digital systems where simultaneous memory access is required for efficient data transfer and processing.

The proposed design implements a parameterized memory architecture with a default data width of 8 bits, address width of 4 bits, and memory depth of 16 locations. Each memory location stores an ECC-protected codeword rather than the raw data alone. The default configuration uses four Hamming parity bits and one overall parity bit, resulting in a 13-bit stored codeword for every 8-bit data word.

To improve the reliability of the memory, an Error Correcting Code (ECC) mechanism based on a SEC-DED style Hamming scheme is incorporated. During write operations, the input data is encoded before being stored in memory. During read operations, the stored codeword is decoded to detect and correct single-bit errors. Double-bit errors are detected as uncorrectable errors, thereby preventing incorrect data correction.

The design also incorporates write-write collision detection and resolution. A collision occurs when both ports attempt to write to the same memory address during the same clock cycle. The design detects this condition and asserts a collision status signal. A deterministic priority mechanism is implemented in which Port A is given priority, ensuring that the memory contents are not affected by an ambiguous simultaneous write.

A Built-In Self-Test (BIST) controller is included to provide an internal mechanism for testing the memory array. The BIST controller is implemented using a finite-state machine and performs a sequence of write and read operations across all memory locations. The BIST sequence verifies the ability of the memory to store and retrieve expected data correctly and reports the test result through `bist_done` and `bist_pass` signals.

The project is verified using a self-checking Verilog testbench. The testbench checks basic Port A and Port B operations, simultaneous dual-port access, collision handling, BIST functionality, single-bit ECC correction, double-bit error detection, and low-power operation. PASS and FAIL counters are used to automatically evaluate the expected and actual results.

2. Objectives

- To design a synchronous dual-port RAM using Verilog HDL at RTL level.
- To implement independent read and write operations through Port A and Port B.
- To implement a parameterized memory architecture with configurable data width, address width, and memory depth.
- To incorporate ECC for single-bit error correction and double-bit error detection.

- To detect and resolve simultaneous write-write collisions between the two ports.
- To implement a BIST controller for automated memory testing.
- To provide a low-power operating mode while retaining existing memory contents.
- To develop a self-checking Verilog testbench for functional verification.
- To validate the design through simulation and synthesis analysis.

3. System Design and Methodology

3.1 Dual-Port RAM Design

The RAM is implemented as a synchronous memory array with two independent access ports. The default configuration consists of **16 memory locations**, selected using a **4-bit address**, with each input/output data word having a width of **8 bits**.

The memory stores ECC-encoded data. With four Hamming parity bits and one overall parity bit, the stored codeword width is 13 bits.

Design Parameters

Parameter	Value
Data Width	8 bits
Address Width	4 bits
Memory Depth	16 locations
ECC Parity Bits	4
Overall Parity	1 bit
Stored Codeword Width	13 bits

These are the default parameters defined in the RTL.

BLOCK DIAGRAM : DUAL PORT RAM WITH COLLISION DETECTION, ECC, BIST & LOW POWER

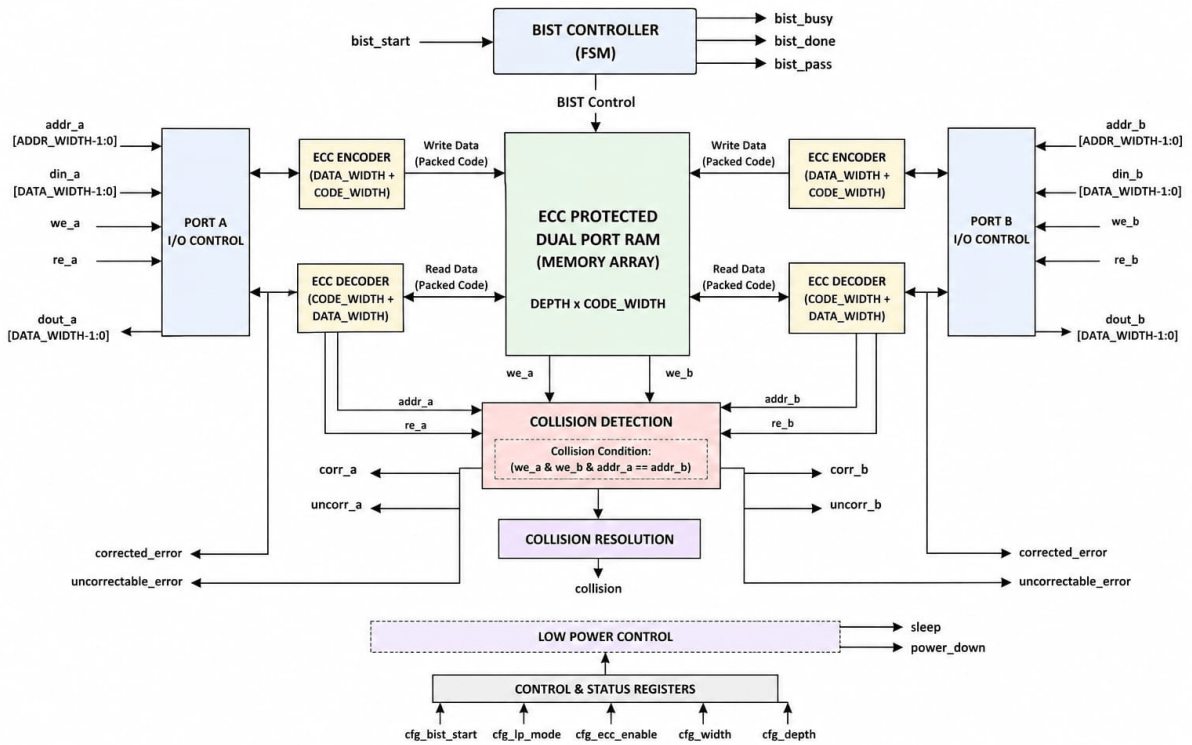


Figure 1: Block Diagram of Dual-Port RAM with ECC, BIST and Collision Handling

3.2 Port A and Port B

The design provides two independent memory access interfaces.

Port A consists of:

- Address input `addr_a`
- Data input `din_a`
- Write enable `we_a`
- Read enable `re_a`
- Data output `dout_a`

Port B consists of:

- Address input `addr_b`
- Data input `din_b`
- Write enable `we_b`
- Read enable `re_b`
- Data output `dout_b`

Both ports operate synchronously with the common clock. The design therefore allows independent access to different memory locations through the two ports during the same clock cycle.

3.3 ECC Implementation

The design uses a SEC-DED style Hamming ECC mechanism. For every 8-bit input data word, four Hamming parity bits and one overall parity bit are generated, producing a 13-bit codeword.

The encoding process is:

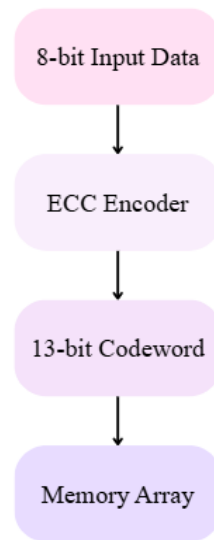


Figure 2: ECC Encoding Process

During a read operation, the stored codeword is passed through the ECC decoder. The decoder calculates the syndrome and overall parity to determine whether an error has occurred.

For a single-bit error, the erroneous bit position is identified and corrected. For a double-bit error, the design raises the `uncorrectable_error` status signal.

3.4 Write-Write Collision Detection

A write-write collision occurs when:

`we_a = 1`

`we_b = 1`

`addr_a = addr_b`

during the same clock cycle.

The collision detection logic compares the write-enable signals and addresses of the two ports.

When the condition is detected:

`collision = 1`

and **Port A is given priority**. The Port A data is ECC encoded and stored in the conflicting memory location.

Example

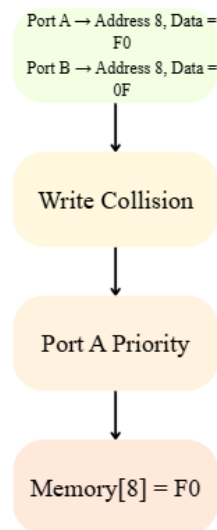


Figure 3: Write - Write Collision Resolution

3.5 BIST Controller

The memory includes a Built-In Self-Test controller implemented using a finite-state machine.

The BIST sequence consists of:

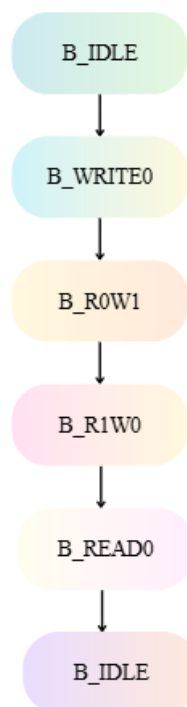


Figure 4: BIST State Sequence

The operations performed are:

- **B_WRITE0:** Write zero to all memory locations.
- **B_R0W1:** Read zero and then write one.
- **B_R1W0:** Read one and then write zero.
- **B_READ0:** Read and verify zero from all locations.

If an unexpected value or uncorrectable error is detected during the sequence, the BIST error flag is set and the final BIST result indicates failure. Otherwise, `bist_pass` is asserted after completion.

3.6 Low-Power Operation

The design also provides a low-power operating mode through the `low_power_mode` input.

When low-power mode is activated:

- RAM read/write activity is stopped.
- BIST activity is stopped.
- Internal state and outputs are held.
- Existing memory contents are retained.
- `low_power_active` indicates that the mode is active.

When the low-power mode is exited, normal operation resumes.

4. Results and Observations

The developed dual-port RAM was functionally verified using a self-checking Verilog testbench. The testbench performs a sequence of directed tests covering the primary memory operations and additional reliability features.

The following test cases were implemented:

Test	Verification	Status
Port A read/write	A5 successfully written/read at address 2	PASS
Port B read/write	3C successfully written/read at address 5	PASS
Simultaneous dual-port	55 and AA correctly read from separate addresses	PASS
Write-write collision	Collision detected; Port A priority	PASS
BIST	BIST completed successfully	PASS

Single-bit ECC	Error corrected and original data recovered	PASS
Double-bit ECC	Error detected as uncorrectable	PASS
Low-power mode	Stored data retained	PASS

Basic Port Operation

Port A was tested by writing A5 to address 2 and subsequently reading the same address. Port B was similarly tested by writing 3C to address 5 and reading it back. The expected values were successfully obtained.

Simultaneous Dual-Port Operation

The design was tested by writing 55 to address 6 through Port A and AA to address 7 through Port B. The two values were subsequently read back from their respective addresses to verify independent simultaneous access.

Collision Verification

For collision testing, both ports attempted to write to address 8 during the same clock cycle:

Port A - F0

Port B - 0F

The collision signal was expected to become 1. Since Port A has priority, a subsequent read of address 8 was expected to return F0. The testbench checks both conditions.

ECC Verification

A single-bit error was deliberately injected into a stored ECC codeword. The decoder successfully recovered the original data and asserted `corrected_error`.

A separate double-bit error was injected into another memory location. The design detected the condition as uncorrectable and asserted `uncorrectable_error`.

BIST Verification

The BIST controller was started through `bist_start`. After completing its memory test sequence, the testbench checked the `bist_pass` signal and monitored the `bist_busy` and `bist_done` status signals.

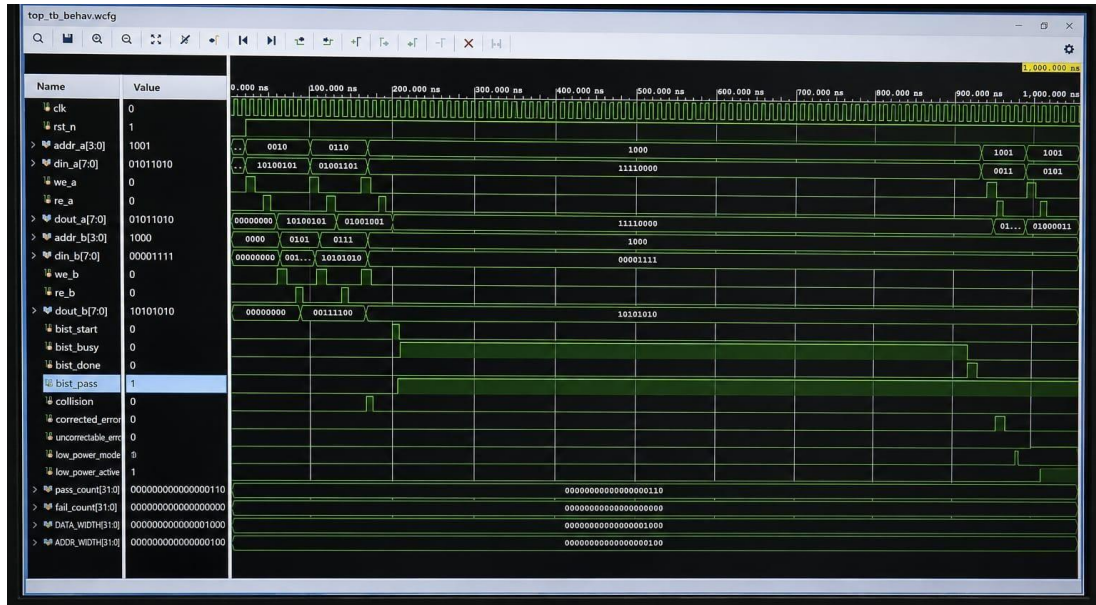


Figure 5: Simulation Waveform

The simulation waveform demonstrates the operation of the dual-port RAM and its associated control and status signals. The waveform is used to verify clock-synchronous read/write operations, collision detection, ECC status signals, BIST operation and low-power control.

5. Synthesis and Power Analysis

The RTL design was synthesized and analyzed using Xilinx Vivado.

The repository contains the elaborated design schematic, synthesized netlist, utilization results and power estimation results.

5.1 Elaborated Design

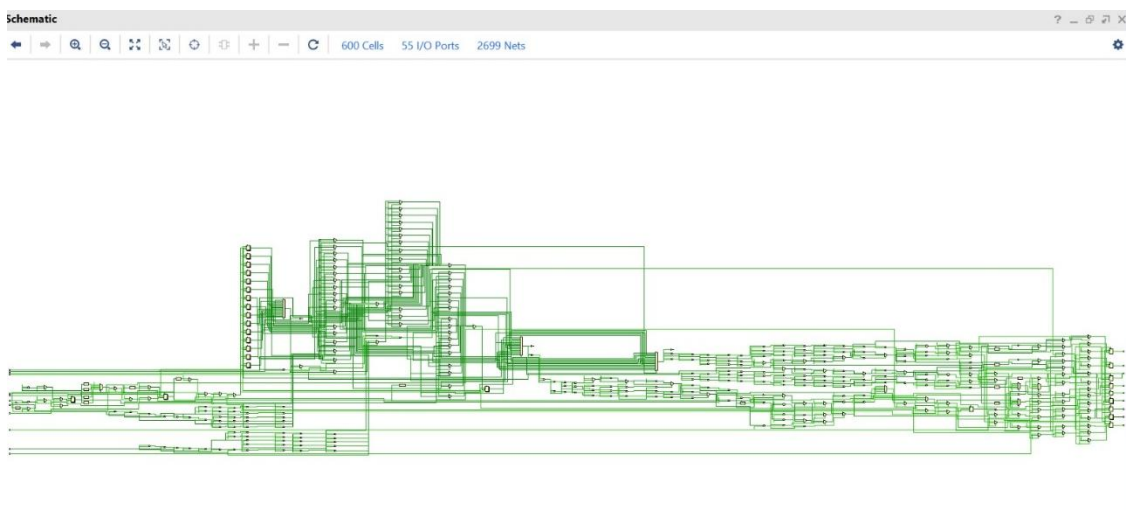


Figure 6: Elaborated RTL Design

This image represents the hardware structure generated from the RTL before final synthesis.

5.2 Synthesized Netlist

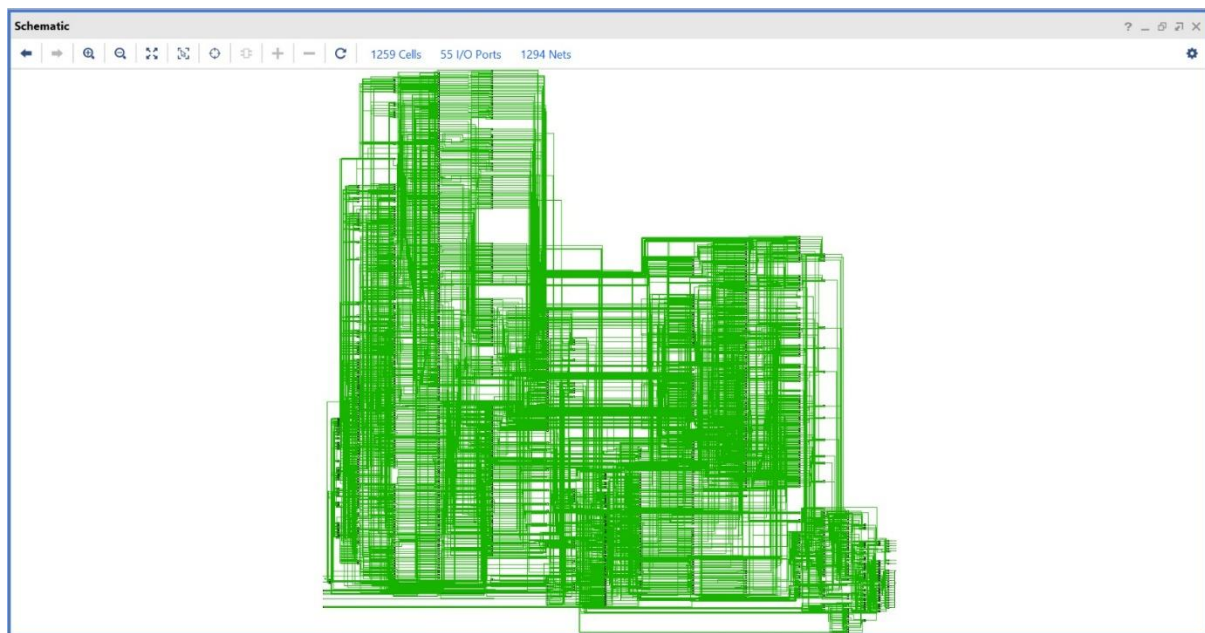


Figure 7: Synthesized Netlist

This represents the synthesized hardware structure generated from the RTL description.

5.3 Resource Utilization

Summary

Resource	Utilization	Available	Utilization %
LUT	775	372352	0.21
FF	239	744704	0.03
IO	55	400	13.75

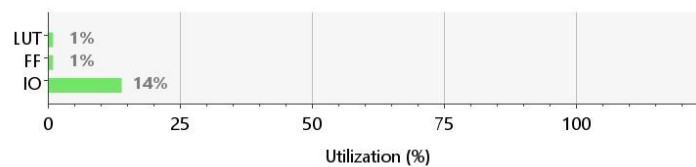


Figure 8: FPGA Resource Utilization

The synthesis report indicates utilization of 775 LUTs, 239 flip-flops and 55 I/O resources. The corresponding reported utilization percentages are 0.21% for LUTs, 0.03% for flip-flops and 13.75% for I/O resources.

5.4 Power Estimation

Power estimation was performed using Xilinx Vivado on the synthesized design. The reported total on-chip power is 10.996 W, consisting of 3.974 W dynamic power and 7.022 W device static power, with the evaluated I/O power reported as 0 W. The reported junction temperature is 100°C at an ambient temperature of 25°C.

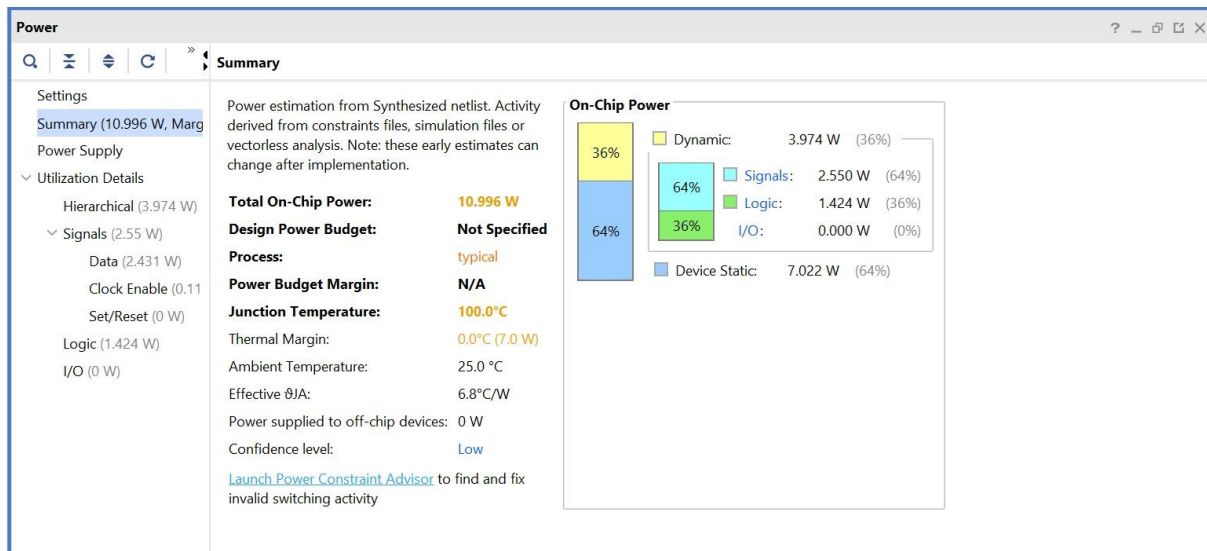


Figure 9: Vivado Power Estimation

Total on-chip power = 10.996 W

Dynamic = 3.974 W

Static = 7.022 W

These values represent an estimated power result obtained from Vivado and are not direct measurements from physical FPGA hardware.

6. Conclusion

The synchronous dual-port RAM was successfully designed at RTL level using Verilog HDL and verified through a self-checking simulation environment. The design supports independent read and write operations through two ports, allowing simultaneous access to different memory locations.

The project extends the basic dual-port RAM architecture with additional reliability and testing mechanisms. ECC provides single-bit error correction and double-bit error detection, while the collision detection mechanism ensures deterministic behavior when both ports attempt to write to the same memory location. A Port A priority scheme is used for resolving write-write collisions.

The integrated BIST controller provides an automated mechanism for testing the memory array through a sequence of write and read operations. The low-power operating mode allows memory activity to be suspended while retaining the existing memory contents.

The testbench verifies basic port operations, simultaneous dual-port access, collision handling, BIST, ECC error handling and low-power operation. The synthesis and power analysis further demonstrate the implementation characteristics of the RTL design. Overall, the project demonstrates the application of RTL design, memory architecture, error correction, built-in testing and functional verification techniques in a digital memory system.

7. Future Scope

The current design can be extended by:

- Increasing the memory depth and data width for larger memory configurations.
- Supporting independent clocks for Port A and Port B to develop an asynchronous dual-port RAM architecture.
- Implementing configurable collision-resolution policies instead of fixed Port A priority.
- Extending ECC capabilities for stronger multi-bit error correction.
- Adding more comprehensive memory fault models to the BIST controller.
- Incorporating SystemVerilog assertions and functional coverage for advanced verification.
- Developing a UVM-based verification environment for scalable verification.
- Implementing the design on an FPGA and comparing synthesized power/resource results with actual hardware measurements.
- Exploring clock gating and additional low-power techniques for reducing dynamic power consumption.

8. Annexure

Source Code

The Verilog HDL source code used for the RTL implementation of the proposed dual-port RAM is provided below. The design code includes the parameterized dual-port RAM architecture, ECC encoder and decoder, write-write collision detection and resolution, BIST controller, and low-power control.

RTL Design: top.v

```
`timescale 1ns/1ps
module top #(
    parameter DATA_WIDTH=8,
    parameter ADDR_WIDTH=4,
    parameter ECC_BITS=4,
    parameter CODE_WIDTH=DATA_WIDTH+ECC_BITS+1,
    parameter DEPTH=(1<<ADDR_WIDTH)
)(
    input clk,
```

```

    input rst_n,

    // Port A
    input [ADDR_WIDTH-1:0] addr_a,
    input [DATA_WIDTH-1:0] din_a,
    input we_a,
    input re_a,
    output reg [DATA_WIDTH-1:0] dout_a,

    // Port B
    input [ADDR_WIDTH-1:0] addr_b,
    input [DATA_WIDTH-1:0] din_b,
    input we_b,
    input re_b,
    output reg [DATA_WIDTH-1:0] dout_b,

    // BIST
    input bist_start,
    output reg bist_busy,
    output reg bist_done,
    output reg bist_pass,

    // Error and collision status
    output reg collision,
    output reg corrected_error,
    output reg uncorrectable_error,

    // Low Power Mode
    input low_power_mode,
    output reg low_power_active
);
reg [CODE_WIDTH-1:0] mem [0:DEPTH-1];
localparam B_IDLE   = 3'b000;
localparam B_WRITE0 = 3'b001;
localparam B_R0W1   = 3'b010;
localparam B_R1W0   = 3'b011;
localparam B_READ0  = 3'b100;
reg [2:0] bist_state;
reg [ADDR_WIDTH-1:0] bist_addr;
reg bist_error;

```

```

reg [DATA_WIDTH-1:0] decoded_data_a;
reg [DATA_WIDTH-1:0] decoded_data_b;
reg decoded_corr_a;
reg decoded_corr_b;
reg decoded_uncorr_a;
reg decoded_uncorr_b;

// =====
// ECC ENCODER
// =====

function [CODE_WIDTH-1:0] ecc_encode;
input [DATA_WIDTH-1:0] data;
reg [CODE_WIDTH-1:0] codeword;
reg parity;
integer pos;
integer data_index;
integer p;
begin
    codeword = {CODE_WIDTH{1'b0}};
    data_index = 0;

    // Place data bits in non-power-of-two positions
    for(pos=1; pos<=DATA_WIDTH+ECC_BITS; pos=pos+1) begin
        if((pos & (pos-1)) != 0) begin
            codeword[pos-1] = data[data_index];
            data_index = data_index + 1;
        end
    end

    // Generate Hamming parity bits
    for(p=0; p<ECC_BITS; p=p+1) begin
        parity = 1'b0;

        for(pos=1; pos<=DATA_WIDTH+ECC_BITS; pos=pos+1) begin
            if((pos & (1<<p)) != 0)
                parity = parity ^ codeword[pos-1];
        end

        codeword[(1<<p)-1] = parity;
    end
end

```

```

    // Overall parity
    parity = 1'b0;
    for(pos=0; pos<DATA_WIDTH+ECC_BITS; pos=pos+1)
        parity = parity ^ codeword[pos];
    codeword[CODE_WIDTH-1] = parity;
    ecc_encode = codeword;
end
endfunction

// =====
// ECC DECODER
// =====

task ecc_decode;
input [CODE_WIDTH-1:0] codeword_in;
output [DATA_WIDTH-1:0] data_out;
output corr;
output uncorr;
reg [CODE_WIDTH-1:0] codeword;
reg parity;
integer syndrome;
integer pos;
integer p;
integer data_index;
begin
    codeword = codeword_in;
    syndrome = 0;
    // Calculate syndrome
    for(p=0; p<ECC_BITS; p=p+1) begin
        parity = 1'b0;
        for(pos=1; pos<=DATA_WIDTH+ECC_BITS; pos=pos+1) begin
            if((pos & (1<<p)) != 0)
                parity = parity ^ codeword[pos-1];
        end
        if(parity)
            syndrome = syndrome | (1<<p);
    end
end
end

```

```

// Calculate overall parity
parity = codeword[CODE_WIDTH-1];
for(pos=0; pos<DATA_WIDTH+ECC_BITS; pos=pos+1)
    parity = parity ^ codeword[pos];
corr = 1'b0;
uncorr = 1'b0;
// Single-bit error
if((parity == 1'b1) && (syndrome != 0)) begin
    if(syndrome <= DATA_WIDTH+ECC_BITS) begin
        codeword[syndrome-1] = ~codeword[syndrome-1];
        corr = 1'b1;
    end
    else begin
        uncorr = 1'b1;
    end
end
// Error in overall parity bit
else if((parity == 1'b1) && (syndrome == 0)) begin
    codeword[CODE_WIDTH-1] = ~codeword[CODE_WIDTH-1];
    corr = 1'b1;
end
// Double-bit error
else if((parity == 1'b0) && (syndrome != 0)) begin
    uncorr = 1'b1;
end
// Extract original data
data_out = {DATA_WIDTH{1'b0}};
data_index = 0;
for(pos=1; pos<=DATA_WIDTH+ECC_BITS; pos=pos+1) begin
    if((pos & (pos-1)) != 0) begin
        data_out[data_index] = codeword[pos-1];
        data_index = data_index + 1;
    end
end
end
end

```

```

endtask

// =====
// MAIN CONTROL
// =====

always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        dout_a <= {DATA_WIDTH{1'b0}};
        dout_b <= {DATA_WIDTH{1'b0}};
        bist_busy <= 1'b0;
        bist_done <= 1'b0;
        bist_pass <= 1'b0;
        collision <= 1'b0;
        corrected_error <= 1'b0;
        uncorrectable_error <= 1'b0;
        low_power_active <= 1'b0;
        bist_state <= B_IDLE;
        bist_addr <= {ADDR_WIDTH{1'b0}};
        bist_error <= 1'b0;
        decoded_data_a <= {DATA_WIDTH{1'b0}};
        decoded_data_b <= {DATA_WIDTH{1'b0}};
        decoded_corr_a <= 1'b0;
        decoded_corr_b <= 1'b0;
        decoded_uncorr_a <= 1'b0;
        decoded_uncorr_b <= 1'b0;
    end
    else begin
        // =====
        // LOW POWER MODE
        // =====
        if(low_power_mode) begin
            // Indicate that low-power mode is active
            low_power_active <= 1'b1;

            // Hold outputs and internal state.

            // No RAM read/write activity.

            // No BIST activity.
        end
    end
end

```



```

        bist_state <= B_R0W1;

    end

    else begin

        bist_addr <= bist_addr + 1'b1;

    end

end

// -----
// READ ZERO / WRITE ONE
// -----

B_R0W1: begin

    ecc_decode(

        mem[bist_addr],

        decoded_data_a,

        decoded_corr_a,

        decoded_uncorr_a

    );

    if(decoded_uncorr_a ||

        (decoded_data_a != {DATA_WIDTH{1'b0}}))

        bist_error <= 1'b1;

    mem[bist_addr] <=

        ecc_encode({DATA_WIDTH{1'b1}});

    if(bist_addr == DEPTH-1) begin

        bist_addr <= {ADDR_WIDTH{1'b0}};

        bist_state <= B_R1W0;

    end

    else begin

        bist_addr <= bist_addr + 1'b1;

    end

end

//-----
// READ ONE / WRITE ZERO
// -----

B_R1W0: begin

    ecc_decode(

        mem[bist_addr],

```

```

        decoded_data_a,
        decoded_corr_a,
        decoded_uncorr_a
    );
    if(decoded_uncorr_a ||
        (decoded_data_a != {DATA_WIDTH{1'b1}}))
        bist_error <= 1'b1;
    mem[bist_addr] <=
        ecc_encode({DATA_WIDTH{1'b0}});
    if(bist_addr == DEPTH-1) begin
        bist_addr <= {ADDR_WIDTH{1'b0}};
        bist_state <= B_READ0;
    end
    else begin
        bist_addr <= bist_addr + 1'b1;
    end
end
// -----
// FINAL READ ZERO
// -----
B_READ0: begin
    ecc_decode(
        mem[bist_addr],
        decoded_data_a,
        decoded_corr_a,
        decoded_uncorr_a
    );
    if(decoded_uncorr_a ||
        (decoded_data_a != {DATA_WIDTH{1'b0}}))
        bist_error <= 1'b1;
    if(bist_addr == DEPTH-1) begin
        bist_busy <= 1'b0;
        bist_done <= 1'b1;
        bist_state <= B_IDLE;
        bist_addr <= {ADDR_WIDTH{1'b0}};
    end
end

```

```

        if(bist_error ||
           decoded_uncorr_a ||
           (decoded_data_a != {DATA_WIDTH{1'b0}}))
            bist_pass <= 1'b0;
        else
            bist_pass <= 1'b1;
        end
    else begin
        bist_addr <= bist_addr + 1'b1;
    end
end
default: begin
    bist_state <= B_IDLE;
    bist_busy <= 1'b0;
    bist_done <= 1'b0;
    bist_pass <= 1'b0;
end
endcase
end

// =====
// NORMAL DUAL-PORT OPERATION
// =====
else begin
    // -----
    // COLLISION HANDLING
    // -----
    if(we_a && we_b && (addr_a == addr_b)) begin
        collision <= 1'b1;
        // Port A has priority
        mem[addr_a] <= ecc_encode(din_a);
    end
    else begin
        if(we_a)
            mem[addr_a] <= ecc_encode(din_a);
        if(we_b)

```

```

        mem[addr_b] <= ecc_encode(din_b);

    end

    // -----
    // PORT A READ
    // -----

    if(re_a) begin
        ecc_decode(
            mem[addr_a],
            decoded_data_a,
            decoded_corr_a,
            decoded_uncorr_a
        );
        dout_a <= decoded_data_a;
        if(decoded_corr_a)
            corrected_error <= 1'b1;
        if(decoded_uncorr_a)
            uncorrectable_error <= 1'b1;
    end

    // -----
    // PORT B READ
    // -----

    if(re_b) begin
        ecc_decode(
            mem[addr_b],
            decoded_data_b,
            decoded_corr_b,
            decoded_uncorr_b
        );
        dout_b <= decoded_data_b;
        if(decoded_corr_b)
            corrected_error <= 1'b1;
        if(decoded_uncorr_b)
            uncorrectable_error <= 1'b1;
    end

end
end

```

```
        end
    end
end
endmodule
```

Verification Testbench: testbench.v

The RTL file contains the parameterized dual-port RAM, ECC encoder/decoder, BIST controller, collision handling and low-power control. The testbench contains the directed tests and automatic PASS/FAIL checking.

GitHub Repository

<https://github.com/YerukondaVenkataLakshmi/Dual-Port-RAM>

The complete project source code and supporting files are available in the above repository.

Simulation Platform

The design can be simulated using EDA Playground or Xilinx Vivado.